Python Image Slideshow App (Using PIL)☐

This project is an Image Slideshow Application built using Python, Tkinter, and the PIL (Pillow) library.

It demonstrates how to use external libraries, handle images, resize them, and display them in a GUI application with time-based transitions.

- This project is an image slideshow application built using Python, Tkinter, and the PIL (Pillow) library to display resized images with time-based transitions.

☐ Important Notes and Terms

☐ Tkinter

Tkinter is Python's built-in GUI library used to create desktop applications with windows, labels, and buttons.

☐ External Library

An external library is a library that is not included by default in Python and must be installed separately.

Example:

☐`pip install pillow`
☐

☐ PIL / Pillow

PIL (Pillow) is an external Python library used for image processing such as opening, resizing, and converting images.

☐ `Image` (PIL Module)

Used to open and resize images.

Example:

```
☐Image.open("image.jpg")
☐
```

☐ ImageTk

Used to convert PIL images into a format that Tkinter can display.

Tkinter cannot show PIL images directly without `ImageTk`.

☐ Module

A module is a Python file that contains reusable code.

Example:

```
☐import tkinter as tk
☐
```

☐ Submodule

A submodule is a smaller part inside a module that provides specific functionality.

Example:

```
☐from PIL import Image, ImageTk
☐
```

☐ tk.Tk()

Creates the main application window.
Without this, the GUI cannot exist.

☐ root.title()

Sets the title of the application window.

☐ root.geometry("900x600")

Sets the window size.

- 900 → width
- 600 → height

☐ Image Paths List

A list is used to store paths of images that will be shown in the slideshow.

Example:

```
☐image_paths = ["img1.jpg", "img2.jpg"]
☐
```

☐ Image Resizing

Images are resized to a fixed size so that:

- All images look uniform
- Slideshow looks clean and professional

Example:

```
☐img.resize((800, 500))
☐
```

☐ `Label` Widget

A Label widget is used as an image frame to display images inside the window.

☐ `image_label.image = photo`

This line keeps a reference to the image.
Without it, the image may disappear due to garbage collection.

☐ Function

A function is a reusable block of code that runs only when called.

Example:

```
☐def start_slideshow():
☐
```

☐ Loop (`for`)

A loop is used to show images one by one in the slideshow.

☐ `time.sleep(seconds)`

Pauses the program for a given number of seconds.
Used here to delay image change so users can view each image.

☐ `root.update()`

Forces the GUI window to refresh immediately and show the updated image.

☐ Button Widget

Buttons are used to trigger actions like starting the slideshow.

Example:

```
☐tk.Button(root, text="Play Slideshow")
☐
```

☐ Event-Driven Programming

The program waits for user actions (button clicks).
Functions run only when an event occurs.

☐ `root.mainloop()`

Starts the event loop.

- Keeps the window open
- Listens for user actions
- Ends when the window is closed

Full Code with Comments

☐ Before running this code

☐pip install pillow

☐

☐# Import tkinter for creating GUI window, buttons, labels
```
import tkinter as tk

# Import time module to pause between images
import time

# Import Image and ImageTk from PIL (external library)
# Image     -> open and resize images
# ImageTk   -> convert images so Tkinter can display them
from PIL import Image, ImageTk


# -------------------- MAIN WINDOW --------------------

# Create the main application window
root = tk.Tk()

# Set the title of the window
root.title("Python Image Slideshow App")

# Set the window size
root.geometry("900x600")


# -------------------- IMAGE PATHS --------------------
# List of image file paths (change paths as per your system)

image_paths = [
    r"C:\Users\YourName\Pictures\image1.jpg",
    r"C:\Users\YourName\Pictures\image2.jpg",
    r"C:\Users\YourName\Pictures\image3.jpg"
]
```

```
# -------------------- IMAGE PROCESSING --------------------

# Set fixed size for all images
image_size = (800, 500)

# List to store resized PIL images
images = []

# Open and resize each image using PIL
for path in image_paths:
    img = Image.open(path)          # Open image
    img = img.resize(image_size)    # Resize image
    images.append(img)              # Store image


# Convert PIL images to Tkinter compatible images
photo_images = []
for img in images:
    photo = ImageTk.PhotoImage(img)
    photo_images.append(photo)


# -------------------- IMAGE DISPLAY AREA --------------------

# Label widget used as image frame
image_label = tk.Label(root)
image_label.pack(pady=20)


# -------------------- SLIDESHOW FUNCTION --------------------

def start_slideshow():
    # Loop through all images
    for photo in photo_images:
        # Set image on label
        image_label.config(image=photo)

        # Keep reference to image (important to avoid image
disappearing)
        image_label.image = photo
```

```
        # Update the GUI to show the image
        root.update()

        # Pause for 2 seconds before showing next image
        time.sleep(2)


# -------------------- BUTTON --------------------

# Button to start the slideshow
play_button = tk.Button(
    root,
    text="Play Slideshow",
    font=("Arial", 14),
    command=start_slideshow
)

# Place button on the window
play_button.pack(pady=10)


# -------------------- RUN APPLICATION --------------------

# Start the event loop and keep window open
root.mainloop()
```

□

- Tkinter → creates the window
- PIL (Pillow) → handles images
- Image → opens & resizes images
- ImageTk → shows images in Tkinter
- time.sleep() → delay between images
- Label → acts as photo frame
- Button → starts slideshow
- mainloop() → keeps app running